AI MATH TUTOR

Image-Based Algebra Solver

Powered by Gemini 2.5 Flash | Streamlit | SymPy

Project Documentation

Academic Year 2025–2026

Team Name: Team 15
Team ID: 69aa7a0dd40347214509364b
Team size: 5

Team Members:

1. Vairam Venkatesh A
2. Swarna Izac A
3. Vasister K
4. Vicky Kumar Mahto B
5. Yugesh V

1. Project Description

AI Math Tutor is an intelligent, image-based algebra problem-solving platform that helps students solve handwritten or printed algebra problems using Optical Character Recognition (OCR) and Google's Gemini 2.5 Flash Vision-Language Model.

Students simply upload a photo of an algebra equation. The system preprocesses the image, extracts mathematical symbols via OCR, converts them to LaTeX using Gemini AI, then solves the equation step by step using SymPy — highlighting any common mistakes along the way.

1.1 Use Case Scenarios

Scenario 1: Image-Based Algebra Problem Recognition

Students upload a photo (JPG/PNG) or capture one via the built-in camera widget. The image is preprocessed and fed through the OCR pipeline.

Output:

- Recognised raw text from OCR engine
- Cleaned mathematical expression
- LaTeX equation rendered in the UI

Scenario 2: Gemini AI — LaTeX Conversion

When the 'Use Gemini AI for LaTeX conversion' checkbox is enabled in the sidebar, the cleaned OCR text is sent to the Gemini 2.5 Flash REST API using the key stored in the .env file.

Output:

- Structured LaTeX representation of the algebra problem
- Rendered via st.latex() in the Streamlit interface

Scenario 3: Step-by-Step Solution Generation

The LaTeX expression is parsed into a SymPy object, solved symbolically, and decomposed into logical intermediate steps.

Output:

- Original equation
- Simplified form
- Rearranged expression (= 0)
- Final computed solution

Scenario 4: Mistake Identification and Feedback

The mistake detection module inspects SymPy objects for common algebraic errors — distribution mistakes, sign errors during transposition, and parsing failures.

Output:

- Mistake alert (st.error) or success confirmation (st.success)
- Human-readable explanation of detected error type

2. Architecture Overview

AI Math Tutor is a modular educational platform built on the Streamlit framework. It processes image-based algebra inputs through multiple layers — image preprocessing, symbol recognition, AI-powered LaTeX conversion, symbolic solving, and mistake detection — then presents results through an interactive web UI.

2.1 Core Technologies

Component	Description
Frontend UI (Streamlit)	Interactive web interface for image upload, LaTeX preview, step-by-step solution display, and mistake alerts. Built with Streamlit sidebar and main canvas layout.
Image Processing Module	Preprocesses uploaded images using OpenCV and PIL — resizing, grayscale conversion, noise reduction, and contrast enhancement via UnsharpMask for improved OCR accuracy.
OCR Layer	Extracts text and mathematical symbols from images using EasyOCR (preferred) with Tesseract as fallback. Includes heuristic math-line scoring to pick best equation candidate.
Vision AI Layer (Gemini 2.5 Flash)	Reads GEMINI_API_KEY from .env and calls the Google Generative Language REST endpoint. Converts cleaned OCR output into structured LaTeX using a fine-tuned prompt in prompts/latex_prompt.txt.
Equation Solver Module (solver.py)	Parses LaTeX into SymPy expressions via parse_latex(); supports both equations (Eq) and pure expressions. Solves linear/quadratic equations and simplifies symbolic forms.
Mistake Detection Module (checker.py)	Inspects SymPy objects for distribution errors, sign-handling issues during transposition, and expression parsing failures, returning human-readable warning messages.
Solution Generator Module	Generates step-by-step solution tuples [(title, latex_step)] by tracking Original → Simplified → Rearranged → Solve stages using SymPy transformations.
Result Display Engine	Renders LaTeX via st.latex(), displays step cards, final answers, and mistake alerts using st.error() / st.success() inside a structured Streamlit UI.

2.2 Project File Structure

File / Folder	Purpose
`app.py`	Streamlit frontend — main entry point. Handles image upload, camera input, OCR pipeline, LaTeX display, and solution rendering.
`vision/ocr.py`	OCREngine class using EasyOCR/Tesseract. llm_convert_to_latex() reads GEMINI_API_KEY and calls Gemini 2.5 Flash REST API.

File / Folder	Purpose
`solver/equation_solver.py`	parse_latex_to_sympy(), solve_equation(), generate_steps() — complete symbolic math pipeline via SymPy.
`checker/mistake_checker.py`	detect_mistakes() — heuristic algebra error detection using SymPy simplification checks.
`utils/image_utils.py`	preprocess_for_ocr(), load_image(), to_bytes() — OpenCV + PIL image preprocessing utilities.
`prompts/latex_prompt.txt`	Prompt template for Gemini API. Uses <<OCR_TEXT>> placeholder, instructs model to return LaTeX only.
`.env`	Environment file. Must contain: GEMINI_API_KEY=your_actual_api_key_here
`requirements.txt`	All dependencies: streamlit, sympy, easyocr, pytesseract, opencv, google-generativeai, python-dotenv, etc.
`samples/`	Test images (eq_1.png to eq_4.png) for OCR validation.
`smoke_test.py / run_smoke_test.py`	Quick end-to-end test suite to validate pipeline without a full UI launch.

3. Pre-Requisites

3.1 Software Requirements

1. Python 3.9 or above
2. Streamlit Framework (pip install streamlit)
3. OpenCV — headless version (opencv-python-headless)
4. EasyOCR (preferred) / Tesseract OCR
5. SymPy — symbolic mathematics library
6. Pillow, NumPy
7. google-generativeai SDK + requests (for Gemini API calls)
8. python-dotenv (for loading .env file)
9. ANTLR4 runtime (antlr4-python3-runtime — required by SymPy LaTeX parser)

3.2 Hardware Requirements

- Laptop or Desktop System
- Camera or Mobile Phone (for capturing algebra problem images)
- Stable Internet Connection (for Gemini API calls)

3.3 API Requirement — Gemini API Key

The application uses Google Gemini 2.5 Flash to convert OCR output into clean LaTeX. You must obtain an API key from Google AI Studio and store it in the .env file.

```
# .env file (project root)
GEMINI_API_KEY=your_actual_api_key_here
```

Steps to obtain the API key:

10. Visit https://makersuite.google.com/app/apikey
11. Sign in with your Google account and accept terms
12. Click "Create API Key" and copy the generated key
13. Paste it into the .env file as shown above
14. The app loads it automatically via load_dotenv() in app.py

4. Project Workflow & Milestones

MILESTONE 1 Gemini AI Initialisation

Establish and configure the core AI infrastructure. The platform relies on Gemini 2.5 Flash (loaded from .env) to interpret algebra problem images and convert OCR output to LaTeX equations.

Activity 1.1 — Set Up Google AI Studio & Enable Gemini API

15. Visit https://makersuite.google.com/app/apikey
16. Sign in using a Google account and accept terms & conditions
17. Click "Create API Key" — copy and store it securely
18. Select Gemini 2.5 Flash — supports multimodal inputs, fast response, suitable for educational/reasoning tasks

Activity 1.2 — Configure .env File

Create a .env file in the ai_math_tutor/ project root:

```
GEMINI_API_KEY=your_actual_api_key_here
```

Install all dependencies:

```
pip install -r requirements.txt
```

The requirements.txt includes: streamlit, easyocr, pytesseract, sympy, opencv-python-headless, google-generativeai, python-dotenv, requests, Pillow, numpy, openai, antlr4-python3-runtime.

Activity 1.3 — Validate Gemini Connectivity

Run the smoke test to verify API connectivity and pipeline integrity without launching the full UI:

```
cd ai_math_tutor
python run_smoke_test.py
```

The smoke test exercises the full pipeline: image preprocessing → OCR extraction → Gemini LaTeX conversion → SymPy solving → mistake detection.

MILESTONE 2 Core Functionality Development

Build the core backend intelligence using Streamlit and Gemini 2.5 Flash. Develop the complete image-to-solution pipeline across four modular Python files.

Activity 2.1 — Image Preprocessing (utils/image_utils.py)

The preprocess_for_ocr() function resizes images to a maximum dimension of 1600px maintaining aspect ratio, converts to grayscale, applies ImageOps.autocontrast(), and sharpens using PIL's UnsharpMask (radius=1, percent=150, threshold=3). The to_bytes() utility serialises PIL images for API transmission.

Activity 2.2 — OCR Extraction (vision/ocr.py)

The OCREngine class initialises EasyOCR (easyocr.Reader) if available, falling back to pytesseract. The extract_math() method runs multi-line scoring to identify the best mathematical candidate from OCR results. Heuristics include: math symbol counting, compacting spaces, and filtering common noise words.

Activity 2.3 — Gemini LaTeX Conversion (vision/ocr.py)

The llm_convert_to_latex() function checks for GEMINI_API_KEY in the environment. If found, it reads prompts/latex_prompt.txt, substitutes the <<OCR_TEXT>> placeholder with cleaned OCR output, and calls the Gemini 2.5 Flash REST endpoint:

```
POST https://generativelanguage.googleapis.com/v1beta/
     models/gemini-2.5-flash:generateContent?key={api_key}

Body: {
  "contents": [{"parts": [{"text": prompt}]}],
  "generationConfig": {"temperature": 0.0, "maxOutputTokens": 512}
}
```

The model returns the LaTeX expression only — no preamble or markdown fences — as defined in the prompt template.

Activity 2.4 — Equation Solver (solver/equation_solver.py)

parse_latex_to_sympy() uses SymPy's parse_latex() to convert LaTeX strings to Eq objects. A fallback heuristic handles common OCR artefacts ('^' → '**', symbol injection for implicit multiplication). solve_equation() calls SymPy's solve() on the detected free symbols. generate_steps() creates a list of (title, latex_step) tuples showing Original → Simplified → Rearranged → Solve progression.

Activity 2.5 — Mistake Detection (checker/mistake_checker.py)

detect_mistakes() inspects the SymPy Eq object for distribution errors (expand() vs original), sign imbalance during transposition (counting '-' vs '+' in simplified combined expression), and expression parsing failures, returning a list of human-readable warning strings.

MILESTONE 3 Backend Integration (app.py)

Integrate all modules into the Streamlit application. app.py is the single backend entry point that wires together the complete pipeline and handles all user interactions.

Activity 3.1 — Application Initialisation

```
from dotenv import load_dotenv
load_dotenv()   # Loads GEMINI_API_KEY from .env

import streamlit as st
from utils.image_utils import preprocess_for_ocr, to_bytes
from vision.ocr import OCREngine, llm_convert_to_latex
from solver.equation_solver import parse_latex_to_sympy, solve_equation,
generate_steps
from checker.mistake_checker import detect_mistakes
```

Activity 3.2 — Sidebar Controls

The Streamlit sidebar provides: file uploader (JPG/PNG), camera capture widget, and a checkbox labelled 'Use Gemini AI for LaTeX conversion (requires GEMINI_API_KEY)'. When the checkbox is enabled, the app calls Gemini; otherwise the cleaned OCR string is used directly as LaTeX.

Activity 3.3 — Processing Pipeline

The main() function follows this sequence:

19. Image is loaded (upload or camera) → displayed in sidebar
20. preprocess_for_ocr() applies OpenCV/PIL preprocessing
21. OCREngine.extract_text() and extract_math() run OCR
22. If Gemini checkbox is on: llm_convert_to_latex() calls Gemini 2.5 Flash
23. parse_latex_to_sympy() converts LaTeX to SymPy object
24. solve_equation(), generate_steps(), detect_mistakes() run in parallel spinner
25. Results rendered: LaTeX display, step cards, final answer, mistake alerts

Activity 3.4 — Error Handling

Each stage is wrapped in try/except blocks. If LaTeX parsing fails, an st.error() is shown and execution stops gracefully. API failures in the Gemini call fall back to the cleaned OCR text. Invalid images are handled by PIL exceptions caught in the preprocessing step.

MILESTONE 4 UI Development

Design and implement the Streamlit frontend for AI Math Tutor. The UI is entirely defined in app.py using Streamlit's widget and layout APIs.

Activity 4.1 — Home Page & Sidebar

- st.title() renders the main heading: 'AI Math Tutor — Image-Based Algebra Solver'
- st.sidebar.header() / st.sidebar.file_uploader() / st.sidebar.camera_input() provide input controls
- Sidebar checkbox toggles Gemini AI LaTeX conversion on/off
- Info message shown when no image is uploaded

Activity 4.2 — Core Display Sections

OCR Output section — st.text_area() shows raw OCR text (height=120).

Cleaned Expression — st.text_input() shows the heuristic math expression.

LaTeX Preview — st.latex() renders the detected/Gemini-produced equation. Falls back to monospace markdown on render failure.

Activity 4.3 — Solution Card

The Solution Card section renders:

- Problem in LaTeX using st.latex()
- Numbered step breakdown: Step 1 — Original, Step 2 — Simplified, Step 3 — Rearranged, Step 4 — Solve
- Each step rendered with st.latex() where possible, st.write() as fallback
- Final answer via st.write(solutions)
- Mistake alert via st.error() or success banner via st.success()

Activity 4.4 — Loading & Feedback

- st.spinner() wraps preprocessing, OCR, and solving phases
- st.info() guides users when no image is uploaded
- Sidebar footer shows environment variable hints

MILESTONE 5 Testing & Optimisation

Validate correctness, reliability, and performance of the full AI Math Tutor system using the included test infrastructure.

Activity 5.1 — Smoke Testing

The project includes smoke_test.py and run_smoke_test.py for rapid end-to-end validation without launching the UI. Sample images (eq_1.png to eq_4.png) in the samples/ folder are used as test inputs.

```
cd ai_math_tutor
python run_smoke_test.py
```

Activity 5.2 — Test Scenarios

The following test types should be validated:

- Simple linear equations — e.g., 2x + 3 = 7
- Multi-step algebraic equations — e.g., 3(x - 2) = 4x + 1
- Equations with fractions and negatives

- Handwritten and printed algebra images
- Edge cases: blurry/low-quality images, incomplete equations, non-algebra images

Activity 5.3 — Quality Evaluation Criteria

Each output is evaluated on:

- Solution Accuracy — correct final answer from SymPy
- Step Clarity — logical, easy-to-follow step breakdown
- LaTeX Accuracy — well-formed mathematical formatting from Gemini
- Mistake Identification — relevance and clarity of detected errors
- Graceful error handling — no crashes, user-friendly fallback messages

Activity 5.4 — Performance Optimisation

- temperature: 0.0 in Gemini API call ensures deterministic LaTeX output
- max_dim=1600 cap in preprocess_for_ocr() balances quality vs. processing speed
- OCR fallback chain (EasyOCR → Tesseract → empty string) prevents pipeline stalls
- LaTeX parse fallback with heuristic sympify() handles malformed OCR output
- All API calls time out after 20 seconds with logged exceptions

5. Running the Application

5.1 Installation

```
# 1. Clone or unzip the project
cd ai_math_tutor

# 2. Install dependencies
pip install -r requirements.txt

# 3. Add your Gemini API key to .env
# GEMINI_API_KEY=your_actual_api_key_here
```

5.2 Launch

```
streamlit run app.py
```

The browser will open automatically at http://localhost:8501. Use the sidebar to upload an algebra image, toggle Gemini AI, and click through the solution steps.

5.3 Using Gemini AI Mode

26. Ensure GEMINI_API_KEY is set in .env
27. Check 'Use Gemini AI for LaTeX conversion' in the sidebar
28. Upload or capture an image of an algebra problem
29. The app will send cleaned OCR output to Gemini 2.5 Flash and render the resulting LaTeX
30. View the step-by-step solution and mistake feedback

6. Conclusion

AI Math Tutor successfully demonstrates the practical application of Generative AI in education by providing an intelligent, interactive, and student-friendly solution for solving algebraic problems from images.

The implemented system integrates: Google Gemini 2.5 Flash (via secured .env API key) for vision-language LaTeX conversion; EasyOCR/Tesseract for symbol recognition; SymPy for accurate symbolic mathematics; and Streamlit for a responsive, user-friendly interface — all within a clean modular architecture.

The application effectively converts algebra problem images into structured LaTeX, solves equations with step-by-step explanations, and identifies common student mistakes. Comprehensive testing through smoke_test.py and sample images validates system reliability.

The project lays a strong foundation for future enhancements including multi-subject support, adaptive learning paths, and personalised student analytics — showcasing the transformative potential of AI-driven tutoring systems in modern education.